University of Asia Pacific
Department of Computer Science and Engineering
CSE 402: Operating System Lab

Lab Report: 02

Report Title: SJF and FCFS Implementation, Comparison with FCFS

Submitted by:

Name : Adika Sultana

Student ID: 23101162

Section : C2

Submitted to:

Atia Rahman Orthi
Lecturer ,
Department of CSE,UAP

Date of Submission: 10/08/2026

1.Problem Statement

Develop a Python program to simulate and compare the First Come First Serve (FCFS) and Shortest Job First (SJF) non-preemptive CPU scheduling algorithms using the following set of processes:

Process ID	Arrival Time (AT)	Burst Time (BT)
P1	3	3
P2	2	5
P3	5	4
P4	1	3
P5	6	2

For each scheduling algorithm, calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for every process.

Also determine the execution sequence, Average Turnaround Time, and Average Waiting Time for both FCFS and SJF. Finally, compare the results of the two algorithms and determine which scheduling algorithm performs better for the given dataset in terms of average waiting time and average turnaround time.

2. Source Code:

```
processes = [
   ["P1", 3, 3],
   ["P2", 2, 5],
   ["P3", 5, 4],
   ["P4", 1, 3],
   ["P5", 6, 2]
]

print("SJF\n")

ready = processes[:]
time = 0
sjf_result = []

while ready:

   available = []

   for p in ready:
      if p[1] <= time:
         available.append(p)
```

```
    if len(available) == 0:
        time = min(ready, key=lambda x: x[1])[1]
        continue

    job = min(available, key=lambda x: x[2])
    ready.remove(job)

    name, at, bt = job

    ct = time + bt
    tat = ct - at
    wt = tat - bt

    sjf_result.append([name, at, bt, ct, tat, wt])

    time = ct

print(f'{"Pd":<6}{"AT":<6}{"BT":<6}{"CT":<6}{"TAT":<6}{"WT":<6}')
for i in sjf_result:
    print(f'{i[0]:<6}{i[1]:<6}{i[2]:<6}{i[3]:<6}{i[4]:<6}{i[5]:<6}')

sjf_avg_tat = sum(i[4] for i in sjf_result) / len(sjf_result)
sjf_avg_wt = sum(i[5] for i in sjf_result) / len(sjf_result)

print("\nExecution Sequence:")
for i in sjf_result:
    print(i[0], end=" ")

print("\n\nAverage TAT =", sjf_avg_tat)
print("Average WT =", sjf_avg_wt)

print("\n\n FCFS\n")

fcfs = sorted(processes, key=lambda x: x[1])

time = 0
fcfs_result = []

for p in fcfs:

    name, at, bt = p

    if time < at:
        time = at

    ct = time + bt
```

```
    tat = ct - at
    wt = tat - bt

    fcfs_result.append([name, at, bt, ct, tat, wt])

    time = ct

print(f'{"Pd":<6}{"AT":<6}{"BT":<6}{"CT":<6}{"TAT":<6}{"WT":<6}')
for i in fcfs_result:
    print(f'{i[0]:<6}{i[1]:<6}{i[2]:<6}{i[3]:<6}{i[4]:<6}{i[5]:<6}')

fcfs_avg_tat = sum(i[4] for i in fcfs_result) / len(fcfs_result)
fcfs_avg_wt = sum(i[5] for i in fcfs_result) / len(fcfs_result)

print("\nExecution Sequence:")
for i in fcfs_result:
    print(i[0], end=" ")

print("\n\nAverage TAT =", fcfs_avg_tat)
print("Average WT =", fcfs_avg_wt)

print("\nCOMPARISON\n")

print(f'{"Algorithm":<12}{"Avg TAT":<12}{"Avg WT":<12}')
print("-"*36)
print(f'{"SJF":<12}{sjf_avg_tat:<12.2f}{sjf_avg_wt:<12.2f}')
print(f'{"FCFS":<12}{fcfs_avg_tat:<12.2f}{fcfs_avg_wt:<12.2f}')

print()

if sjf_avg_tat < fcfs_avg_tat:
    print("SJF has lower Average Turnaround Time.")

if sjf_avg_wt < fcfs_avg_wt:
    print("SJF has lower Average Waiting Time.")

print("\nResult: SJF is Better than FCFS.")
```

GitHub: https://github.com/Adika2003/CSE-402-Operating-System-Lab

3. Result:

```
C:\Users\lab\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\lab\PycharmProjects\PythonProject\23101162_LAB_3.py
========== SJF ==========

Pd    AT    BT    CT    TAT   WT
P4    1     3     4     3     0
P1    3     3     7     4     1
P5    6     2     9     3     1
P3    5     4     13    8     4
P2    2     5     18    16    11

Execution Sequence:
P4 P1 P5 P3 P2

Average TAT = 6.8
Average WT = 3.4


========== FCFS ==========

Pd    AT    BT    CT    TAT   WT
P4    1     3     4     3     0
P2    2     5     9     7     2
P1    3     3     12    9     6
```

```
P4    1     3     4     3     0
P2    2     5     9     7     2
P1    3     3     12    9     6
P3    5     4     16    11    7
P5    6     2     18    12    10

Execution Sequence:
P4 P2 P1 P3 P5

Average TAT = 8.4
Average WT = 5.0

========== COMPARISON ==========

Algorithm   Avg TAT     Avg WT
-----------------------------------
SJF         6.80        3.40
FCFS        8.40        5.00

SJF has lower Average Turnaround Time.
SJF has lower Average Waiting Time.

Result: SJF is Better than FCFS.
```

4. Why SJF is better than FCFS ?

In this problem, SJF is better than FCFS because it selects the shortest available process first**,** which helps reduce the overall waiting time of the processes. In FCFS, processes are executed only according to their arrival time, so a process with a longer burst time may cause shorter processes to wait for a long time. For example, P2 has a burst time of 5 units and is executed before some shorter processes in FCFS simply because it arrived earlier. In SJF, shorter processes such as P1 and P5 are executed before longer processes whenever they are available. As a result, more processes finish earlier and the average waiting time and turnaround time become lower. In this dataset, the average waiting time of SJF is 3.4**,** while FCFS has 5.0**,** and the average turnaround time of SJF is 6.8**,** compared with 8.4 for FCFS. Therefore, SJF gives better performance than FCFS for this particular set of processes.

5. Discussion:

In this problem, both FCFS and non-preemptive SJF scheduling algorithms were applied to the same set of processes to compare their performance. The results show that FCFS executes processes strictly according to their arrival time, while SJF selects the shortest available process based on burst time. Because of this difference, SJF is able to complete shorter processes earlier and reduce unnecessary waiting in the ready queue. For the given dataset, FCFS produced an average turnaround time of 8.4 units and an average waiting time of 5.0 units**,** whereas SJF produced a lower average turnaround time of 6.8 units and a lower average waiting time of 3.4 units**.** This indicates that SJF is more efficient for this particular process set in terms of both waiting time and turnaround time. However, SJF may delay processes with larger burst times, as seen with P2, which is executed later despite arriving early. Therefore, although SJF gives better average performance in this problem, FCFS is simpler and provides fair service based on arrival order, while SJF prioritizes efficiency over strict arrival-order fairness.